

Buổi 6

Segment tree, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hào

Tài liệu một file cho buổi segment tree, bản Python. Gồm bốn phần: kiến thức, mười đề bài, gợi ý và lời giải, và mười code mẫu đầy đủ.

Đề bài và bộ test dùng chung với bản C++ ở `../../topics/06-segment-tree/`. Cùng một đề, cùng một bộ test, khác ngôn ngữ, nên bạn so sánh được hai lối viết cạnh nhau. Cái khác không nằm ở thuật toán mà ở **hình dạng code mà ngôn ngữ chịu được**, và mục 3 dưới đây là chỗ nói thẳng chuyện đó bằng số đo.

1. Mục lục

Phần	Nội dung
I	Kiến thức, mười một mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Phần I. Kiến thức

1. Vì sao cần segment tree

Cho một dãy và hai loại thao tác trộn lẫn nhau: sửa một phần tử, và hỏi một đại lượng trên một đoạn. Tính lại từ đầu mỗi lần hỏi mất $O(n)$ cho một truy vấn, nên $2 \cdot 10^5$ truy vấn trên dãy $2 \cdot 10^5$ phần tử là $4 \cdot 10^{10}$ phép tính. Không ngôn ngữ nào cứu được con số đó.

Fenwick tree của buổi trước hạ được xuống $O(\log n)$, nhưng nó tính truy vấn đoạn bằng **hiệu hai tổng tiền tố**, nên phép toán bắt buộc phải có nghịch đảo. Cộng có nghịch đảo là trừ. Lấy giá trị lớn nhất thì không có gì đóng vai nghịch đảo cả: biết max của $[1..7]$ và max của $[1..3]$ không suy ra được max của $[4..7]$.

Segment tree không cần nghịch đảo. Nó chỉ cần phép hợp **kết hợp được**, tức $(a \circ b) \circ c = a \circ (b \circ c)$. Điều kiện nhẹ đó mở ra max, min, gcd, và cả những nút mang bốn con số như bài G.

2. Cấu trúc, và vì sao bản Python viết lặp chứ không viết đệ quy

Bản C++ của buổi này dùng cây đệ quy trên mảng $4n$: nút v phủ đoạn $[tl, tr]$, hai con là $2v$ và $2v+1$. Đó là hình dạng dễ đọc nhất, và với C++ nó cũng đủ nhanh.

Bản Python dùng cây **lặp** trên mảng $2 \cdot \text{size}$ với size là lũy thừa của hai đầu tiên không nhỏ hơn n :

- lá thứ i của dãy nằm ở ô $\text{size} + i$,
- cha của nút v là $v \gg 1$, hai con là $2v$ và $2v+1$,
- những lá dư từ $\text{size} + n$ tới $2 \cdot \text{size} - 1$ là **lá đệm**, phải mang phần tử trung hòa.

Truy vấn đoạn $[l, r]$ đi từ hai đầu lên:

```
l += size
r += size
res = TRUNG_HOA
while l < r:
    if l & 1:
        res = hop(res, t[l]); l += 1
    if r & 1:
        r -= 1; res = hop(res, t[r])
    l >>= 1
    r >>= 1
```

Lý do đổi hình dạng không phải là sở thích. Xem mục tiếp theo.

3. Ba lần đo, và con số quyết định cách viết cả buổi

Cùng một máy, cùng một bộ test lớn nhất của kho ($n = q = 2 \cdot 10^5$), cùng một bài C (cộng đoạn, tổng đoạn):

Cách viết	Thời gian
đệ quy, đúng hình dạng bản C++	3,8 s
lặp, có hàm con ap, day, keo	2,9 s
lặp, gỡ hàm con thành mã thẳng	2,7 s

Giới hạn của chủ đề này bên Python là 6 s, nên cả ba đều lọt. Nhưng nếu giữ giới hạn 2 s của bản C++ thì **cách dễ đọc nhất là cách duy nhất trượt**.

Đó không phải một khuyết tật của Python cần né. Đó là giá của trình thông dịch: mỗi lời gọi hàm, mỗi lần tra tên vượt qua biên hàm, đều trả bằng thời gian chạy. Trong một vòng lặp chạy vài triệu lượt, khoản đó lớn hơn chính phép tính bạn muốn làm.

Và đây là con số ngược chiều, đo trên cùng máy đó:

Bài	C++ của kho	Python của kho	
A, không lazy, nặng vào ra	0,49 s	0,31 s	Python nhanh hơn 1,6 lần
C, có lazy, nặng tính toán	0,61 s	2,67 s	Python chậm 4,4 lần

Bài A Python thắng, vì nó đọc cả khối dữ liệu một lần rồi tách, còn c++ trả giá cho từng số. Bài C Python thua, vì phần việc thật nằm trong vòng lặp của trình thông dịch.

Kết luận dùng được: không có một tỉ lệ nào giữa hai ngôn ngữ để nhân vào rồi suy ra. Có bài Python nhanh hơn, có bài chậm gấp bốn. Cách duy nhất biết là đo, và đó là lý do nhánh này có công cụ tools/qa-thoi-gian.py đo từng bài trên từng test.

4. Đọc dữ liệu: dòng code quan trọng nhất của cả buổi

```
data = sys.stdin.buffer.read().split()
```

Đọc hết đầu vào thành một khối byte rồi tách một lần. Sau đó đi bằng con trỏ p và ép kiểu khi cần: `int(data[p])`.

Ba thứ **không** được dùng ở buổi này:

Đừng	Vì sao
<code>input()</code>	mỗi lời gọi là một lượt vào ra có khoá, với $2 \cdot 10^5$ dòng nó một mình ăn hết giới hạn
<code>sys.stdin.readline()</code> trong vòng lặp	khá hơn <code>input()</code> nhiều nhưng vẫn thua đọc một khối
<code>print(x)</code> từng dòng	gom vào một list rồi <code>'\n'.join</code> và ghi một lần

So sánh `data[p] == b'1'` chứ không `int(data[p]) == 1`: đỡ được một lần ép kiểu trên mỗi truy vấn, và `data` là list các đối tượng bytes nên về phải phải có tiền tố b.

5. Phép hợp và phân tử trung hòa

Mỗi bài cần trả lời hai câu trước khi gõ dòng nào:

- Phép hợp hai đoạn kề nhau là gì?
- Phần tử trung hòa của phép hợp đó là gì?

Câu thứ hai là chỗ mất điểm nhiều nhất, vì chọn sai vẫn cho kết quả đúng trên phần lớn dữ liệu.

Phép hợp	Trung hòa	Chọn nhầm thành	Sai khi nào
tổng	0	không có	không có
max	$-(1 \ll 62)$	0	mọi phần tử trong đoạn đều âm
min	$1 \ll 62$	0	mọi phần tử trong đoạn đều dương
gcd	0	1	mọi truy vấn, vì $\text{gcd}(1, x) = 1$

Trung hòa xuất hiện ở **hai** chỗ: giá trị của lá đệm, và giá trị khởi tạo của biến tích lũy trong truy vấn. Trong cây lập, chỉ chỗ thứ hai là quan trọng, và đây là một khác biệt thật giữa cây lập với cây đệ quy chứ không phải chi tiết vụn.

Cây đệ quy trả về phần tử trung hòa mỗi khi đoạn của nút **không giao** với đoạn truy vấn, nên giá trị đệm chảy thẳng vào kết quả. Cây lập không bao giờ đọc một nút không nằm trọn trong đoạn hỏi, nên **lá đệm**

không tham gia kết quả. Điều này đã được kiểm chứ không phải suy đoán: đặt lá đệm của bài B bằng 1 thay vì 0 rồi đối chiếu vết cạy 3.000 ca ngẫu nhiên với n từ 1 tới 9, kết quả lệch **0 trên 3.000**.

Nhưng đặt **biến tích lũy** bằng 1 thì sai ngay truy vấn đầu tiên, vì $\gcd(1, x) = 1$ với mọi x . Nên câu hỏi phải đặt đúng chỗ: trung hòa của bạn nằm ở dòng nào? Với cây lập, dòng đáng soi là $\text{res} = \dots$ chứ không phải dòng khởi tạo mảng.

Python không có `INT_MIN` nên viết `-(1 << 62)`. Số nguyên Python không tràn, nên hằng này không phải là "nhỏ nhất có thể" mà chỉ là "đủ nhỏ so với $|a_i| \leq 10^9$ ". Đó là một khác biệt có lợi: bài D bên C++ phải lo tổng $2 \cdot 10^{14}$ vượt `int`, bên Python chuyện đó không tồn tại.

6. Nút mang nhiều hơn một con số

Bài G cần bốn số một nút, bài I cần hai. Bên C++ người ta gói vào một `struct`. Bên Python **đừng** gói vào tuple hay class: mỗi lần ghép là một lần cấp phát đối tượng, và cấp phát là phần đắt nhất của vòng lặp trong CPython.

Dùng nhiều list song song thay vào đó:

```
tong = [0] * (2 * size)
pre = [NEG] * (2 * size)
suf = [NEG] * (2 * size)
tot = [NEG] * (2 * size)
```

Xấu hơn về mặt hình thức, nhưng nó là lỗi viết mà ngôn ngữ này chịu được.

7. Phép hợp không giao hoán

Max, min, gcd, tổng đều giao hoán, nên truy vấn lặp gom các mảnh theo thứ tự nào cũng ra cùng kết quả. Phép ghép của bài G **không** giao hoán: đoạn con vắt qua ranh giới cần hậu tố của mảnh **trái** nối với tiền tố của mảnh **phải**.

Nên trong truy vấn lặp phải giữ hai bộ tích lũy riêng, một cho các mảnh nhất từ đầu trái, một cho các mảnh nhất từ đầu phải, và chỉ ghép hai bộ đó với nhau ở bước cuối cùng, đúng thứ tự trái rồi phải. Gom chung một biến là lỗi cho kết quả đúng trên mọi truy vấn nằm gọn trong một mảnh, tức là phần lớn test nhỏ.

8. Đi xuống cây

Bài F hỏi chỉ số nhỏ nhất i với $a_i \geq x$. Cách hiển nhiên là chặt nhị phân trên đáp án, bên trong mỗi bước gọi một truy vấn max: $O(\log^2 n)$.

Cách đáng học là **đi xuống cây**, $O(\log n)$. Nút gốc đã biết max của cả dãy, nên nếu nó nhỏ hơn x thì đáp án là -1 . Ngược lại đi xuống, và luôn bước sang con **trái** khi con trái còn chứa nổi một phần tử đủ lớn:

```
v = 1
while v < size:
    v <= 1
    if t[v] < x:
        v += 1
return v - size + 1
```

Lá đệm mang `-(1 << 62)` nên đường đi không bao giờ lạc vào chúng. Đổi thứ tự thành thử con phải trước thì vẫn tìm ra một vị trí thỏa mãn, chỉ là không phải vị trí nhỏ nhất, mà đề hỏi vị trí *nhỏ nhất*.

9. Lazy propagation và ba loại thẻ

Khi thao tác sửa cả một đoạn, không thể đi tới từng lá. Ý tưởng: dừng ở những nút phủ trọn đoạn cần sửa, ghi lên nút một **thẻ chờ** mang nghĩa "mọi phần tử bên dưới tôi còn phải nhận khoản này", và cập nhật giá trị của chính nút đó ngay.

Buổi này có đúng ba loại thẻ, và chúng khác nhau ở cách **gộp hai thẻ**:

Loại	Bài	Gộp thẻ cũ với thẻ mới	Áp thẻ x vào nút
cộng	C, H	cộng dồn: $lz += x$	tổng: $t += x * w$; min: $t += x$
gán	D, E	đề: $lz = x$	tổng: $t = x * w$; max: $t = x$
đảo	J	lật: $lz ^= 1$	$t = w - t$

Hai cột bên phải là hai chỗ chép nhầm nhiều nhất trong cả buổi.

- Cột phải, dòng "cộng": cây **tổng** nhân với bề rộng w , cây **min** thì không. Cộng cùng một x vào mọi phần tử làm giá trị nhỏ nhất dịch đúng x , không nhân gì cả. Bài H chính là bài C đổi một dòng này, và để nguyên nhân w thì kết quả vẫn đúng trên mọi đoạn rộng một.
- Cột phải, dòng "gán": cây **tổng** thành $x * w$, cây **max** thành x . Bài E là bài D đổi đúng dòng này.

Còn một bẫy riêng của thẻ gán: **v có thể bằng 0**, và gán 0 là một phép gán thật. Nên $lz[v] != 0$ không được phép mang nghĩa "đang có thẻ chờ"; phải có một mảng cờ riêng $co[v]$. Dùng chính giá trị làm cờ cho một chương trình đúng trên gần như mọi test và sai đúng lúc có ai đó gán về không.

10. Thứ tự đẩy thẻ trong cây lập

Cây đệ quy đẩy thẻ xuống ngay trước khi đi vào con. Cây lập không đi xuống, nên phải đẩy **trước**, dọc hai đường biên, từ trên xuống:

```
for v in (1, r - 1):
    for h in range(LOG, 0, -1):
        u = v >> h
        # push u's pending tag down into its two children
```

Đẩy từ LOG xuống 1, tức từ gần gốc xuống gần lá, không được đảo chiều: một thẻ ở gốc phải xuống tới nút giữa trước khi nút giữa đẩy tiếp thẻ của nó.

Và khi kéo giá trị ngược lên sau khi sửa, giá trị của một nút là hợp của hai con **cộng thêm thẻ của chính nó**, vì hai con chưa nhận thẻ ấy:

```
t[v] = t[2*v] + t[2*v+1] + lz[v] * w[v]
```

Quên về cuối là lỗi chỉ hiện ra khi có hai thao tác sửa lồng nhau trên cùng một nhánh.

11. Cạm bẫy Python, ngoài phần thuật toán

Cạm bẫy	Hậu quả
<code>sys.setrecursionlimit</code> lớn mà vẫn đệ quy sâu	Python tràn stack thật của hệ điều hành và chết không truy vết, báo về là RE chứ không phải RecursionError
/ thay //	/ luôn cho số thực, và số thực mất chính xác từ khoảng $9 \cdot 10^{15}$, tức là ngay trong tầm dữ liệu bài D
ghép chuỗi kết quả bằng <code>s += ...</code> trong vòng lặp	$O(n^2)$ vì chuỗi Python bất biến; dùng list rồi <code>join</code>
<code>t = [[0] * m] * n</code>	n hàng cùng trỏ vào một list; sửa hàng 0 là sửa tất cả
gọi <code>max(a, b)</code> trong vòng lặp nóng	lời gọi hàm đắt; viết <code>a if a > b else b</code>

Ba dòng cuối không phải chuyện thẩm mỹ. Trong bài C chúng là khoản chênh giữa 2,7 s và 3,8 s.

12. Bộ khung tự hỏi khi gặp đề mới

- Đại lượng cần hỏi là gì, và ghép hai đoạn kề nhau thì nó tính thế nào?

- Một con số mỗi nút có đủ không? Không đủ thì thiếu những gì?
- Phần tử trung hòa là gì? Thử đặt nó vào một đoạn toàn âm, toàn dương, và một đoạn rỗng.
- Phép ghép có giao hoán không? Không thì truy vấn lặp phải tách trái với phải.
- Thao tác sửa chạm một phần tử hay cả đoạn? Cả đoạn thì thẻ thuộc loại nào trong ba loại ở mục 9?
- Áp thẻ vào một nút thì giá trị nút thành gì? Có nhân với bề rộng không?
- Giá trị θ có phải một giá trị hợp lệ của thẻ không? Có thì cần mảng cờ riêng.
- Ước lượng số phép tính. Trên $2 \cdot 10^5$ truy vấn mà mỗi truy vấn $\log n \approx 18$ bước, Python làm khoảng $4 \cdot 10^6$ lượt vòng lặp, cỡ 2 tới 3 giây. Nếu ước lượng ra hơn 10^7 lượt thì phải nghĩ lại trước khi gõ.

Phần II. Mười bài tập

Đề PDF	Ở đây	Kỹ thuật chính	File nộp
A	Bài 1	segment tree cơ bản, max	bai01.py
B	Bài 2	đổi phép hợp thành gcd	bai02.py
C	Bài 3	lazy cộng đoạn, tổng đoạn	bai03.py
D	Bài 4	lazy gán đoạn, tổng đoạn	bai04.py
E	Bài 5	lazy gán đoạn, max đoạn	bai05.py
F	Bài 6	đi xuống cây	bai06.py
G	Bài 7	nút mang bốn giá trị	bai07.py
H	Bài 8	lazy cộng đoạn, min đoạn	bai08.py
I	Bài 9	max kèm số lần xuất hiện	bai09.py
J	Bài 10	lazy đảo bit	bai10.py

Giới hạn thời gian **6 giây** cho mọi bài, gấp ba bản C++ vì đây là nhánh Python; xem mục I.3 để biết con số 6 từ đâu ra. Dữ liệu vào lên tới 5 MB. Tự chấm:

```
python3 chay.py topics/06-segment-tree/solutions/bai01.py
python3 chay.py topics/06-segment-tree --tat-ca
```

Trong mọi bài, chỉ số của mảng đánh số từ 1, và thao tác loại 1 là cập nhật còn loại 2 là truy vấn.

Bài A. Giá trị lớn nhất trên đoạn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 i v : gán $a_i = v$.
- 2 l r : in giá trị lớn nhất trong đoạn từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	5
1 5 2 4 3	2
2 1 5	4
1 2 0	
2 1 3	
2 4 5	

Lưu ý

Fenwick tree của buổi trước không giải được bài này. Hãy tự trả lời vì sao trước khi viết code, và cẩn thận với giá trị trả về khi đoạn của nút không giao với đoạn truy vấn.

Giải thích

Giá trị lớn nhất của cả dãy là 5. Sau khi gán phần tử thứ hai bằng 0, đoạn 1 tới 3 gồm 1, 0, 2 nên lớn nhất là 2, còn đoạn 4 tới 5 gồm 4 và 3 nên lớn nhất là 4.

Bài B. Ước chung lớn nhất trên đoạn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên dương và q thao tác:

- 1 i v : gán $a_i = v$.
- 2 l r : in ước chung lớn nhất của các phần tử từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)
- q dòng thao tác ($1 \leq v \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 3 12 18 24 6 9 2 1 3 1 2 5 2 1 3	6 1

Bài này dùng đúng khung code của bài 1, chỉ khác một dòng. Nhưng phần tử trung hòa thì khác, hãy nghĩ kỹ nó là gì.

Lưu ý

Bài này dùng đúng khung code của bài A, chỉ khác một dòng. Nhưng phần tử trung hòa thì khác hẳn, và chọn nhầm nó là lỗi sai gần như mọi lúc.

Giải thích

Ước chung lớn nhất của 12, 18, 24 là 6. Sau khi gán phần tử thứ hai bằng 5, ba số thành 12, 5, 24 và ước chung lớn nhất tụt xuống 1.

Bài C. Cộng đoạn, tổng đoạn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 $l \ r \ v$: cộng thêm v vào mọi phần tử từ l tới r .
- 2 $l \ r$: in tổng các phần tử từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	15
1 2 3 4 5	45
2 1 5	25
1 2 4 10	
2 1 5	
2 2 3	

Tổng có thể đạt $2 \cdot 10^{14}$ sau các phép cộng.

Lưu ý

Tổng có thể đạt $2 \cdot 10^{14}$ sau các phép cộng. Khi áp thẻ lazy vào một nút, hãy nghĩ kỹ xem giá trị nút tăng bao nhiêu.

Giải thích

Tổng ban đầu là 15. Cộng 10 vào ba vị trí từ 2 tới 4 làm tổng tăng thêm 30 nên thành 45. Đoạn 2 tới 3 lúc này là $12 + 13 = 25$.

Bài D. Gán đoạn, tổng đoạn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 $l \ r \ v$: gán mọi phần tử từ l tới r bằng v .
- 2 $l \ r$: in tổng các phần tử từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	15
1 2 3 4 5	27
2 1 5	14
1 2 4 7	
2 1 5	
2 3 4	

Chú ý v có thể bằng 0, và đó là một phép gán thật sự chứ không phải không làm gì.

Lưu ý

Giá trị v có thể bằng 0, và đó là một phép gán thật sự chứ không phải không làm gì. Điều này quyết định cách bạn đánh dấu một nút đang có thể chờ.

Giải thích

Tổng ban đầu là 15. Gán 7 cho ba vị trí từ 2 tới 4 cho dãy 1, 7, 7, 7, 5 với tổng 27, và đoạn 3 tới 4 là $7 + 7 = 14$.

Bài E. Gán đoạn, giá trị lớn nhất trên đoạn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 $l \ r \ v$: gán mọi phần tử từ l tới r bằng v .
- 2 $l \ r$: in giá trị lớn nhất trong đoạn từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	5
1 2 3 4 5	2
2 1 5	2
1 3 5 2	
2 1 5	
2 4 5	

Bài này dùng đúng thể lazy của bài 4. Nhưng khi áp thể vào một nút thì giá trị nút tính khác, hãy cẩn thận chỗ đó.

Lưu ý

Bài này dùng đúng thể lazy của bài D. Nhưng khi áp thể vào một nút thì giá trị nút được tính khác, và chép nguyên bản cũ sang mà quên sửa là lỗi rất dễ mắc.

Giải thích

Giá trị lớn nhất ban đầu là 5. Gán 2 cho ba vị trí cuối cho dãy 1, 2, 2, 2, 2, nên lớn nhất của cả dãy và của đoạn 4 tới 5 đều bằng 2.

Bài F. Vị trí đầu tiên đủ lớn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 i v : gán $a_i = v$.
- 2 x : in chỉ số nhỏ nhất i sao cho $a_i \geq x$, hoặc -1 nếu không có.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v|, |x| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	4
1 3 2 5 4	2
2 4	-1
1 2 9	
2 4	
2 100	

Có cách $O(\log^2 n)$ và có cách $O(\log n)$. Cả hai đều kịp, nhưng cách thứ hai đáng học.

Lưu ý

Có cách $O(\log^2 n)$ và có cách $O(\log n)$. Cả hai đều kịp, nhưng cách thứ hai đáng học vì nó dùng lại được ở nhiều bài khó hơn. Chú ý đề hỏi chỉ số *nhỏ nhất*.

Giải thích

Trong dãy 1, 3, 2, 5, 4, phần tử đầu tiên không nhỏ hơn 4 nằm ở vị trí 4. Sau khi gán phần tử thứ hai bằng 9 thì vị trí đầu tiên thỏa mãn là 2. Không phần tử nào đạt 100 nên truy vấn cuối in -1 .

Bài G. Đoạn con liên tiếp có tổng lớn nhất

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 i v : gán $a_i = v$.
- 2 l r : xét mọi đoạn con liên tiếp **không rỗng** nằm trong đoạn từ l tới r , in tổng lớn nhất trong số đó.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 3	4
-1 2 -3 4 -5	9
2 1 5	
1 3 3	
2 1 5	

Giải thích: ban đầu đoạn con tốt nhất là 4. Sau khi gán $a_3 = 3$, đoạn 2 3 4 cho tổng 9.

Lưu ý

Một con số trong mỗi nút là không đủ. Hãy nghĩ xem khi ghép hai đoạn kề nhau thì đoạn con tốt nhất của đoạn hợp có thể nằm ở những đâu. Đoạn con phải không rỗng, nên nếu mọi phần tử đều âm thì đáp án là phần tử lớn nhất.

Bài H. Cộng đoạn, giá trị nhỏ nhất trên đoạn

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 $l\ r\ v$: cộng thêm v vào mọi phần tử từ l tới r .
- 2 $l\ r$: in giá trị nhỏ nhất trong đoạn từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	1
5 3 8 1 9	3
2 1 5	3
1 4 4 10	
2 1 5	
2 1 2	

Bài này dùng đúng thể lazy của bài 3. Nhưng có một dòng phải viết khác, hãy tìm ra nó.

Lưu ý

Bài này dùng đúng thể lazy của bài C, nhưng có một dòng phải viết khác. Nếu bạn vừa làm xong bài C và chép cấu trúc sang, hãy tìm ra dòng đó trước khi nộp.

Giải thích

Giá trị nhỏ nhất ban đầu là 1 ở vị trí 4. Cộng 10 vào đúng vị trí đó làm nó thành 11, nên nhỏ nhất của cả dãy chuyển thành 3. Đoạn 1 tới 2 gồm 5 và 3 nên nhỏ nhất cũng là 3.

Bài I. Giá trị lớn nhất và số lần xuất hiện

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 i v : gán $a_i = v$.
- 2 l r : in giá trị lớn nhất trong đoạn từ l tới r , và số lần giá trị đó xuất hiện trong đoạn.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng gồm hai số cách nhau bởi dấu cách.

Ví dụ

Dữ liệu vào	Kết quả
5 3 1 3 3 2 3 2 1 5 1 1 5 2 1 5	3 3 5 1

Lưu ý

Khi ghép hai nút có ba trường hợp chứ không phải hai. Bỏ sót một trường hợp thì giá trị lớn nhất vẫn luôn đúng và chỉ số đếm sai, nên lỗi rất khó bị nghi ngờ.

Giải thích

Dãy 1, 3, 3, 2, 3 có giá trị lớn nhất là 3 và nó xuất hiện ba lần. Sau khi gán phần tử đầu bằng 5, giá trị lớn nhất là 5 và nó chỉ xuất hiện một lần.

Bài J. Đảo bit trên đoạn, đếm số 1

Giới hạn thời gian: 6 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số, mỗi số bằng 0 hoặc 1, và q thao tác:

- 1 $l\ r$: đảo mọi bit trong đoạn từ l tới r , tức 0 thành 1 và 1 thành 0.
- 2 $l\ r$: in số lượng số 1 trong đoạn từ l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số, mỗi số là 0 hoặc 1
- q dòng thao tác

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	3
1 0 1 0 1	4
2 1 5	1
1 2 4	
2 1 5	
2 2 3	

Lưu ý

Thẻ lazy của bài này khác hẳn hai loại trước. Hãy tự trả lời: đảo hai lần liên tiếp thì được gì, và điều đó nói gì về cách gộp hai thẻ với nhau?

Giải thích

Dãy 1, 0, 1, 0, 1 có ba số 1. Đảo đoạn từ 2 tới 4 cho dãy 1, 1, 0, 1, 1 với bốn số 1, và trong đoạn 2 tới 3 lúc này chỉ còn một số 1.

Phần III. Gợi ý và lời giải

Đọc từng tăng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự	Riêng với Python
A	phần tử trung hòa của max đặt bằng 0	không có
B	phần tử trung hòa của gcd đặt bằng 1	<code>math.gcd</code> có sẵn, đừng tự viết
C	thể cộng quên nhân với số lá mà nút phủ	đề quy ở bài này là 3,8 s, xem I.3
D	dùng chính giá trị làm cờ, mất mọi phép gán về 0	không có
E	quên đẩy thẻ xuống trong hàm cập nhật	không có
F	đi xuống cây theo hướng phải trước	không có
G	quên đoạn con vắt qua ranh giới	đừng gói nút vào tuple, dùng list song song
H	chép nhầm dòng áp thẻ từ cây tổng sang cây min	không có
I	quên trường hợp hai bên bằng nhau	không có
J	thẻ đảo gán bằng 1 thay vì lật	không có

Bài A. Giá trị lớn nhất trên đoạn

Gợi ý 1. Fenwick tính truy vấn đoạn bằng hiệu hai tổng tiền tố, nên nó cần phép toán có nghịch đảo. Lấy giá trị lớn nhất có nghịch đảo không?

Gợi ý 2. Không có, nên phải dùng cấu trúc ghép đoạn từ các nút phủ trọn, tức segment tree.

Lời giải. Cây lập như mục I.2: lá thứ i ở ô size $+ i$, dựng cây bằng một vòng từ size $- 1$ xuống 1. Sửa một phần tử là gán vào lá rồi đi ngược lên cập nhật cha. Truy vấn đoạn đi từ hai đầu lên. Mỗi thao tác $O(\log n)$.

Cái bẫy nằm ở phần tử trung hòa. Lá đệm và biến tích lũy của truy vấn phải mang giá trị không làm hỏng phép max, tức $-(1 \ll 62)$, không phải 0. Dùng 0 thì mọi dữ liệu dương vẫn đúng, và chỉ sai khi truy vấn rơi vào vùng toàn âm. Test cỡ tối đa của bài này có giá trị âm chính vì vậy.

Một tối ưu nhỏ đáng giữ. Khi đi ngược lên sau lúc sửa, nếu giá trị của một nút cha không đổi thì mọi nút phía trên nó cũng không đổi, nên break ngay. Trên dữ liệu thật nó cắt được phần lớn quãng đường lên gốc.

Python. Viết `a if a > b else b` thay `max(a, b)` trong vòng dựng cây và vòng đi lên. Đo được khoảng 15 phần trăm trên bài này, và lý do là mỗi `max()` là một lời gọi hàm thật.

Bài B. Ước chung lớn nhất trên đoạn

Gợi ý 1. Đây là đúng khung của bài A, chỉ khác phép hợp.

Gợi ý 2. Ước chung lớn nhất của một tập không đổi khi thêm vào một số 0.

Gợi ý 3. Vậy phần tử trung hòa của gcd là gì?

Lời giải. Thay max bằng `math.gcd` và thay trung hòa $-(1 \ll 62)$ bằng 0. Không đổi gì khác.

Vì sao trung hòa là 0 chứ không phải 1. Vì $\gcd(0, x) = x$ với mọi x , còn $\gcd(1, x) = 1$ với mọi x . Đặt 1 thì mọi đáp án bị kéo tụt về 1.

Lỗi ấy lộ ở đâu, và không lộ ở đâu. Đặt 1 vào **biến tích lũy** `res` thì sai ngay truy vấn đầu tiên, vì $\gcd(1, x) = 1$. Nhưng đặt 1 vào **lá đệm** thì với cây lập không sai gì cả: truy vấn chỉ đọc những nút nằm trọn trong đoạn hỏi, mà lá đệm thì không bao giờ nằm trong đó. Đã kiểm bằng đối chiếu vết cận 3.000 ca $n \leq 9$, lệch 0 trên 3.000.

Đây là chỗ bản Python khác bản C++ về bản chất chứ không phải về cú pháp. Cây đệ quy của bản C++ trả trung hòa cho mọi nút **không giao** với đoạn hỏi, nên ở đó giá trị đệm đi thẳng vào đáp án và chọn nhầm là chết thật. Chép nguyên lời cảnh báo từ tài liệu C++ sang đây là dạy một điều đúng về một cấu trúc khác.

Python. `math.gcd` là hàm C, nhanh hơn hẳn một hàm Euclid tự viết bằng Python. Từ 3.5 nó nhận số không âm; ở bài này $a_i \geq 1$ nên không phải lo dấu.

Bài C. Cộng đoạn, tổng đoạn

Gợi ý 1. Không thể đi tới từng lá trong một thao tác cộng đoạn. Dừng ở đâu thì hợp lý?

Gợi ý 2. Dừng ở những nút phủ trọn đoạn, và ghi lên nút một lời hứa rằng phần dưới còn phải nhận.

Gợi ý 3. Khi áp lời hứa ấy vào một nút, tổng của nút tăng bao nhiêu?

Lời giải. Mỗi nút giữ ba thứ: $t[v]$ là tổng của đoạn **đã tính cả thể của chính nó**, $lz[v]$ là khoản hai con còn nợ, và $w[v]$ là số phần tử thật mà đoạn phủ. Áp thể x vào nút v làm $t[v] += x * w[v]$ và $lz[v] += x$.

Vì sao phải nhân với w . Cộng x vào mỗi phần tử trong một đoạn có w phần tử làm tổng tăng $x * w$. Quên nhân thì đáp án vẫn đúng với mọi đoạn rộng một, tức đúng trên mọi test nhỏ viết tay.

Vì sao cần w riêng chứ không tính từ chỉ số. Vì cây đệm lên lũy thừa của hai, nên một nút gần biên phủ cả lá thật lẫn lá đệm. w đếm lá thật nên nó tự động bằng 0 ở phần đệm và mọi phép nhân ở đó vô hại.

Thứ tự trong một thao tác. Đẩy thể xuống dọc hai đường biên trước, từ gần gốc xuống gần lá; rồi áp thể lên các nút phủ trọn; rồi kéo giá trị ngược lên hai đường biên với công thức $t[v] = t[2v] + t[2v+1] + lz[v] * w[v]$. Quên về cuối là lỗi chỉ hiện khi có hai thao tác sửa lồng nhau trên cùng một nhánh.

Python. Đây là bài của mục I.3. Bản đệ quy chạy 3,8 s, bản lập có hàm con 2,9 s, bản gỡ hàm con 2,7 s. Nếu bạn viết bản đệ quy và thấy nó vẫn AC, đó là vì giới hạn của nhánh này là 6 s chứ không phải vì đệ quy ổn.

Không phải lo. Tổng đạt $2 \cdot 10^{14}$, bên C++ đó là lý do phải dùng `long long`. Số nguyên Python không tràn nên bài này mất đúng cái bấy ấy.

Bài D. Gán đoạn, tổng đoạn

Gợi ý 1. Thẻ ở đây không cộng dồn với thẻ cũ. Nó làm gì?

Gợi ý 2. Nó đề lên. Vậy làm sao biết một nút đang có thẻ hay không?

Lời giải. Thẻ mang nghĩa "trở thành", nên thẻ mới **đề** thẻ cũ: $lz[v] = x$ chứ không $+=$. Áp thẻ x vào nút làm $t[v] = x * w[v]$.

Cái bấy. v có thể bằng 0, và gán 0 là một phép gán thật sự. Nên $lz[v] != 0$ không được dùng làm dấu hiệu "đang có thẻ chờ". Phải có mảng cờ riêng `co[v]`. Bỏ qua chuyện này cho một chương trình đúng trên gần như mọi test và sai đúng lúc có người gán về không.

Một chỗ khác với bài C. Khi kéo giá trị ngược lên, nếu nút đang có thẻ chờ thì giá trị của nó đã đúng rồi, đừng tính lại từ hai con, vì hai con vẫn mang dữ liệu cũ. Nên vòng kéo lên phải hỏi `if not co[v]` trước khi tính.

Python. Dùng `co = [False] * (2 * size)` chứ đừng dùng `lz = [None] * (2 * size)` với `None` làm cờ. Hai cách cùng đúng, nhưng `is not None` chậm hơn đọc một phần tử bool, và trong vòng đẩy thẻ nó chạy $18 * 2$ lượt mỗi thao tác.

Bài E. Gán đoạn, giá trị lớn nhất trên đoạn

Gợi ý 1. Thử y hệt bài D. Chỉ một dòng phải viết khác.

Gợi ý 2. Áp thử x vào một nút thì giá trị lớn nhất của nút thành bao nhiêu?

Lời giải. Thành x , không nhân với gì cả. Gán cùng một giá trị cho mọi phần tử của đoạn thì giá trị lớn nhất chính là giá trị ấy.

Vì sao đây là bài riêng chứ không phải bài D viết lại. Vì chép bài D sang và chỉ đổi phép hợp từ tổng thành max là chưa đủ: dòng $t[v] = x * w[v]$ phải thành $t[v] = x$. Chương trình quên đổi dòng đó cho kết quả đúng với mọi đoạn rộng một, và sai ngay ở đoạn rộng hai.

Lá đệm. Cây max nên lá đệm mang $-(1 \ll 62)$ như bài A, và vì ở đây thử gán không bao giờ chạm vào chúng (đoạn cần gán luôn nằm trong $[1, n]$) nên chúng giữ nguyên giá trị ấy suốt.

Bài F. Vị trí đầu tiên đủ lớn

Gợi ý 1. Có cách $O(\log^2 n)$ bằng chặt nhị phân trên đáp án, bên trong gọi truy vấn max. Nó kịp. Nhưng có cách tốt hơn.

Gợi ý 2. Nút gốc đã biết giá trị lớn nhất của cả dãy. Nếu nó nhỏ hơn x thì sao?

Gợi ý 3. Nếu nó không nhỏ hơn x thì đi xuống. Bước sang con nào trước?

Lời giải. Đi xuống cây, $O(\log n)$. Nếu $t[1] < x$ thì in -1 . Ngược lại đặt $v = 1$ rồi lặp: nhân đôi v , và nếu con trái $t[v] < x$ thì $v += 1$ để sang con phải. Khi v chạm tầng lá thì đáp án là $v - \text{size} + 1$.

Vì sao phải thử con trái trước. Đề hỏi chỉ số **nhỏ nhất**. Thử con phải trước vẫn tìm ra một vị trí thỏa mãn, chỉ là không phải vị trí nhỏ nhất, và test có đúng một phần tử đủ lớn sẽ không bắt được lỗi này.

Lá đệm. Chúng mang $-(1 \ll 62)$ và đề cho $|x| \leq 10^9$, nên đường đi không bao giờ lạc vào phần đệm. Nếu đặt trung hòa bằng 0 thì với x âm, đường đi có thể dừng ở một lá đệm và in ra một chỉ số lớn hơn n .

Python. Bài này là bài nhanh nhất của cả buổi, đo 187 ms, vì mỗi truy vấn chỉ đi đúng một đường từ gốc xuống lá chứ không gom mảnh từ hai đầu.

Bài G. Đoạn con liên tiếp có tổng lớn nhất

Gợi ý 1. Một con số trong nút không đủ. Hãy nghĩ xem khi ghép hai đoạn kề nhau, đoạn con tốt nhất của đoạn hợp có thể nằm ở đâu.

Gợi ý 2. Ba khả năng: nằm trọn bên trái, nằm trọn bên phải, hoặc **vắt qua ranh giới**.

Gợi ý 3. Trường hợp thứ ba cần hậu tố tốt nhất của bên trái và tiền tố tốt nhất của bên phải. Vậy nút phải giữ bốn số.

Lời giải. Mỗi nút giữ tong, pre (tiền tố tốt nhất), suf (hậu tố tốt nhất), tot (đoạn con tốt nhất). Ghép trái L với phải R:

```
tong = L.tong + R.tong
pre  = max(L.pre, L.tong + R.pre)
suf  = max(R.suf, R.tong + L.suf)
tot  = max(L.tot, R.tot, L.suf + R.pre)
```

Đoạn con phải không rỗng, nên pre, suf, tot khởi tạo bằng $-(1 \ll 62)$ chứ không bằng 0, và tong khởi tạo bằng 0. Kiểm lại: ghép phần tử trung hòa ấy với một nút R bất kỳ phải ra đúng R. Nếu để pre = 0 thì một dãy toàn âm cho đáp án 0, trong khi đáp án đúng là phần tử lớn nhất.

Chỗ riêng của cây lập. Phép ghép này **không giao hoán**. Truy vấn lập nhất mảnh từ cả hai đầu, nên phải giữ hai bộ tích lũy riêng, một cho các mảnh nhất ở đầu trái và một cho các mảnh nhất ở đầu phải, rồi chỉ ghép hai bộ đó ở bước cuối, theo đúng thứ tự trái rồi phải. Gom chung vào một biến cho kết quả đúng với mọi truy vấn nằm gọn trong một mảnh, tức là phần lớn test nhỏ.

Python. Bốn list song song tong, pre, suf, tot thay vì một list các tuple bốn phần tử. Mỗi lần ghép mà tạo một tuple là một lần cấp phát, và trên $2 \cdot 10^5$ truy vấn thì khoản cấp phát ấy lớn hơn chính phép tính.

Bài H. Cộng đoạn, giá trị nhỏ nhất trên đoạn

Gợi ý 1. Thử hết bài C.

Gợi ý 2. Nhưng có đúng một dòng phải viết khác. Áp thể x vào một nút thì giá trị nhỏ nhất của nút tăng bao nhiêu?

Lời giải. Tăng đúng x , **không nhân với bề rộng**. Cộng cùng một x vào mọi phần tử của đoạn thì giá trị nhỏ nhất dịch đúng x .

Đây là dòng để chép nhằm nhất cả buổi. Bạn vừa viết xong bài C với $t[v] += x * w[v]$, chép cấu trúc sang, đổi $+$ thành $\min$, và để nguyên phép nhân. Kết quả đúng trên mọi đoạn rộng một.

Lá đệm. Cây min nên chúng mang $+(1 \ll 62)$. Chúng vẫn nhận thể như mọi nút khác, nhưng vì giá trị xuất phát quá lớn nên không bao giờ thắng một phần tử thật với $|a_i| \leq 10^9$ và $|v| \leq 10^9$ trên $2 \cdot 10^5$ thao tác.

Kéo lên. $t[v] = \min(t[2v], t[2v+1]) + lz[v]$, cộng chứ không nhân, cùng lý do.

Bài I. Giá trị lớn nhất và số lần xuất hiện

Gợi ý 1. Nút giữ hai số. Khi ghép hai nút, có mấy trường hợp?

Gợi ý 2. Ba, không phải hai.

Lời giải. Nút giữ mx và dem . Ghép: trái lớn hơn thì lấy nguyên bên trái; phải lớn hơn thì lấy nguyên bên phải; **bằng nhau thì lấy giá trị ấy với tổng hai số đếm**.

Vì sao lỗi này sống dai. Bỏ sót trường hợp thứ ba thì giá trị lớn nhất vẫn đúng **mọi lúc**, vì nó chỉ là max của hai con. Chỉ số đếm sai, và sai theo hướng nhỏ hơn thật. Người viết nhìn cột đầu tiên thấy đúng nên không nghi ngờ cột thứ hai.

Lá đệm. $mx = -(1 \ll 62)$ với $dem = 0$. Số đếm bằng 0 là cần thiết: nếu để $dem = 1$ thì hai lá đệm ghép nhau cho $dem = 2$, và một nút gần biên sẽ cộng số đếm mà ấy vào đáp án. Chúng không bao giờ bằng một phần tử thật vì $|a_i| \leq 10^9$.

Phép ghép này giao hoán, nên truy vấn lặp gom mảnh theo thứ tự nào cũng được, khác hẳn bài G.

Bài J. Đảo bit trên đoạn, đếm số 1

Gợi ý 1. Đảo hai lần thì được gì? Điều đó nói gì về cách gộp hai thể?

Gợi ý 2. Đảo là nghịch đảo của chính nó, nên thể chỉ cần một bit và phép gộp là phép lật.

Gợi ý 3. Khi áp thể vào một nút, số lượng số 1 trở thành bao nhiêu?

Lời giải. Thể là một bit, gộp bằng $lz[v] \wedge 1$ chứ không $lz[v] = 1$. Áp thể vào nút làm $t[v] = w[v] - t[v]$, vì đảo mọi bit biến số lượng số 1 thành số lượng số 0.

Vì sao = 1 là sai. Vì nó làm mất tính "đảo hai lần bằng không đảo". Hai thao tác đảo chồng lên cùng một nút phải triệt tiêu nhau; gán bằng 1 thì chúng cộng dồn thành một lần đảo, và đáp án sai đúng ở những nút bị chạm chần lần.

Kéo lên. Giống bài C về hình thức nhưng công thức khác: $t[v]$ bằng tổng hai con, rồi **nếu** $lz[v]$ bật thì lật kết quả ấy: $t[v] = w[v] - s$. Không phải cộng thêm như thể cộng.

Một chi tiết của đề. Cả hai loại thao tác đều có dạng $l \ r$, ba số, không có v . Nên phần đọc dữ liệu của bài này gọn hơn chín bài kia, và phần đẩy thể xuống dọc hai đường biên làm chung cho cả hai loại trước khi rẽ nhánh.

Lá đệm. w của chúng bằng 0, nên $t = w - t = 0 - 0 = 0$ và phép lật không sinh ra số 1 mà nào.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau, vì mã in ra giấy không được bộ test nào chấm nên nó là chỗ dễ lạc hậu nhất trong cả kho.

Bài A. Giá trị lớn nhất trên đoạn

```
# Problem A: range maximum with point assignment.
# A Fenwick tree cannot do this. A range query there is a difference of two prefixes,
# which needs an inverse operation, and maximum has none. A segment tree only needs the
# operation to be associative, so it covers max, min, gcd and much more.
#
# Python note. The C++ version of this topic uses a recursive tree over 4*n nodes. Here
# the tree is ITERATIVE over a power of two, because a Python function call costs far
# more than the work it performs at one node. Layout: leaves sit at size..size+n-1, the
# parent of node v is v>>1, and its children are 2v and 2v+1.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    size = 1
    while size < n:
        size <<= 1
    # Padding leaves must never win a maximum, so they hold the neutral element of max.
    NEG = -(1 << 62)
    t = [NEG] * (2 * size)
    for i in range(n):
        t[size + i] = int(data[p]); p += 1
    for v in range(size - 1, 0, -1):
        a = t[2 * v]; b = t[2 * v + 1]
        t[v] = a if a > b else b

    out = []
    for _ in range(q):
        loai = data[p]
        if loai == b'1':
            i = int(data[p + 1]); v = int(data[p + 2]); p += 3
            j = size + i - 1
            t[j] = v
            j >>= 1
            while j:
                a = t[2 * j]; b = t[2 * j + 1]
                moi = a if a > b else b
                # Stop as soon as a parent stops changing: the rest of the path up to
                # the root cannot change either.
                if t[j] == moi:
                    break
                t[j] = moi
                j >>= 1
            else:
                l = int(data[p + 1]) - 1 + size
                r = int(data[p + 2]) + size
                p += 3
                res = NEG
                while l < r:
```

```

        if l & 1:
            if t[l] > res: res = t[l]
            l += 1
        if r & 1:
            r -= 1
            if t[r] > res: res = t[r]
        l >>= 1; r >>= 1
    out.append(res)

sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài B. Ước chung lớn nhất trên đoạn

```

# Problem B: range gcd with point assignment.
# Same skeleton as problem A, one line different: max becomes gcd. The part that is NOT
# the same is the neutral element. For max it is minus infinity; for gcd it is ZERO,
# because gcd(0, x) == x for every x. Padding the tree with 1 instead would force every
# answer down to 1 and the bug would only show on inputs whose length is not a power of
# two, which is exactly the kind of bug small tests miss.
import sys
from math import gcd

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    size = 1
    while size < n:
        size <<= 1
    t = [0] * (2 * size)      # 0 is the neutral element of gcd
    for i in range(n):
        t[size + i] = int(data[p]); p += 1
    for v in range(size - 1, 0, -1):
        t[v] = gcd(t[2 * v], t[2 * v + 1])

    out = []
    for _ in range(q):
        loai = data[p]
        if loai == b'1':
            i = int(data[p + 1]); v = int(data[p + 2]); p += 3
            j = size + i - 1
            t[j] = v
            j >>= 1
            while j:
                moi = gcd(t[2 * j], t[2 * j + 1])
                if t[j] == moi:
                    break
                t[j] = moi
                j >>= 1
        else:
            l = int(data[p + 1]) - 1 + size
            r = int(data[p + 2]) + size
            p += 3
            res = 0
            while l < r:

```

```

        if l & 1:
            res = gcd(res, t[l]); l += 1
        if r & 1:
            r -= 1; res = gcd(res, t[r])
        l >>= 1; r >>= 1
    out.append(res)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài C. Cộng đoạn, tổng đoạn

```

# Problem C: add on a range, sum on a range. The first lazy problem of the topic.
# A node holds the sum of its segment. A pending tag on a node means "every element
# below me still has to receive this much", and the node's own sum ALREADY counts it.
# Applying tag x to a node whose segment has w elements raises its sum by x*w, which is
# why every node carries the width of its segment.
#
# Python note, measured on this repo's largest test (n = q = 2*10^5):
#   recursive, the shape the C++ version uses ..... 3.8 s
#   iterative with helper functions ..... 2.9 s
#   iterative with the helpers written out by hand ..... 2.7 s   <- this file
# The recursive shape is the clearest to read and it is the one that does not fit.
# That is not a Python quirk to work around, it is the cost of an interpreter: every
# call, and every name lookup that crosses a function boundary, is paid at run time.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    LOG = max(1, (n - 1).bit_length())
    size = 1 << LOG
    t = [0] * (2 * size)      # sum of the segment, pending tag included
    lz = [0] * (2 * size)     # pending "add" still owed to the children
    w = [0] * (2 * size)      # how many real elements the segment holds
    for i in range(n):
        t[size + i] = int(data[p]); w[size + i] = 1; p += 1
    for v in range(size - 1, 0, -1):
        t[v] = t[2 * v] + t[2 * v + 1]
        w[v] = w[2 * v] + w[2 * v + 1]

    out = []
    ghi = out.append
    for _ in range(q):
        loai = data[p]
        if loai == b'1':
            l = int(data[p + 1]) - 1 + size
            r = int(data[p + 2]) + size
            x = int(data[p + 3]); p += 4
            # Push every tag on the two boundary paths down first, so the nodes we are
            # about to touch carry no debt to anyone below them.
            for v in (l, r - 1):
                for h in range(LOG, 0, -1):
                    u = v >> h
                    z = lz[u]

```



```

        if z:
            c = 2 * u; t[c] += z * w[c]; lz[c] += z
            c += 1; t[c] += z * w[c]; lz[c] += z
            lz[u] = 0
    l0 = l; r0 = r
    while l < r:
        if l & 1:
            t[l] += x * w[l]; lz[l] += x; l += 1
        if r & 1:
            r -= 1; t[r] += x * w[r]; lz[r] += x
        l >>= 1; r >>= 1
    # Rebuild the two boundary paths. A parent's sum is the sum of its children
    # PLUS its own pending tag, which the children have not received yet.
    for v in (l0, r0 - 1):
        v >>= 1
        while v:
            t[v] = t[2 * v] + t[2 * v + 1] + lz[v] * w[v]
            v >>= 1
    else:
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        p += 3
        for v in (l, r - 1):
            for h in range(LOG, 0, -1):
                u = v >> h
                z = lz[u]
                if z:
                    c = 2 * u; t[c] += z * w[c]; lz[c] += z
                    c += 1; t[c] += z * w[c]; lz[c] += z
                    lz[u] = 0
            s = 0
            while l < r:
                if l & 1:
                    s += t[l]; l += 1
                if r & 1:
                    r -= 1; s += t[r]
                l >>= 1; r >>= 1
            ghi(s)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài D. Gán đoạn, tổng đoạn

```

# Problem D: assign on a range, sum on a range.
# The tag changes meaning: it is no longer "add this", it is "become this". Two
# consequences, and the second is the trap the statement warns about.
# 1. Tags do not accumulate, a newer tag REPLACES an older one.
# 2. v may be 0, so the tag value 0 is a real assignment. "lz[v] != 0" therefore
# cannot mean "a tag is pending" and a separate flag array is required. Using 0 as
# the empty marker gives a program that is right on almost every test and wrong
# exactly when someone assigns zero.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0

```

```

n = int(data[p]); q = int(data[p + 1]); p += 2

LOG = max(1, (n - 1).bit_length())
size = 1 << LOG
t = [0] * (2 * size)
lz = [0] * (2 * size)
co = [False] * (2 * size)    # is a tag pending on this node
w = [0] * (2 * size)
for i in range(n):
    t[size + i] = int(data[p]); w[size + i] = 1; p += 1
for v in range(size - 1, 0, -1):
    t[v] = t[2 * v] + t[2 * v + 1]
    w[v] = w[2 * v] + w[2 * v + 1]

out = []
ghi = out.append
for _ in range(q):
    loai = data[p]
    if loai == b'1':
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        x = int(data[p + 3]); p += 4
        for v in (l, r - 1):
            for h in range(LOG, 0, -1):
                u = v >> h
                if co[u]:
                    z = lz[u]
                    c = 2 * u; t[c] = z * w[c]; lz[c] = z; co[c] = True
                    c += 1; t[c] = z * w[c]; lz[c] = z; co[c] = True
                    co[u] = False
        l0 = l; r0 = r
        while l < r:
            if l & 1:
                t[l] = x * w[l]; lz[l] = x; co[l] = True; l += 1
            if r & 1:
                r -= 1; t[r] = x * w[r]; lz[r] = x; co[r] = True
            l >>= 1; r >>= 1
        for v in (l0, r0 - 1):
            v >>= 1
            while v:
                if not co[v]:
                    t[v] = t[2 * v] + t[2 * v + 1]
                    v >>= 1
        else:
            l = int(data[p + 1]) - 1 + size
            r = int(data[p + 2]) + size
            p += 3
            for v in (l, r - 1):
                for h in range(LOG, 0, -1):
                    u = v >> h
                    if co[u]:
                        z = lz[u]
                        c = 2 * u; t[c] = z * w[c]; lz[c] = z; co[c] = True
                        c += 1; t[c] = z * w[c]; lz[c] = z; co[c] = True
                        co[u] = False
            s = 0
            while l < r:
                if l & 1:
                    s += t[l]; l += 1
                if r & 1:
                    r -= 1; s += t[r]
                l >>= 1; r >>= 1
            ghi(s)

```

```

sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài E. Gán đoạn, giá trị lớn nhất trên đoạn

```

# Problem E: assign on a range, maximum on a range.
# Exactly the tag of problem D, and that is the trap. Copying problem D across and
# changing only the query is wrong in one place: applying tag x to a node makes its
# SUM equal x*w, but makes its MAXIMUM equal x, with no width factor at all. The
# resulting program agrees with the right answer on every segment of width one.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    LOG = max(1, (n - 1).bit_length())
    size = 1 << LOG
    NEG = -(1 << 62)
    t = [NEG] * (2 * size)
    lz = [0] * (2 * size)
    co = [False] * (2 * size)
    for i in range(n):
        t[size + i] = int(data[p]); p += 1
    for v in range(size - 1, 0, -1):
        a = t[2 * v]; b = t[2 * v + 1]
        t[v] = a if a > b else b

    out = []
    ghi = out.append
    for _ in range(q):
        loai = data[p]
        if loai == b'1':
            l = int(data[p + 1]) - 1 + size
            r = int(data[p + 2]) + size
            x = int(data[p + 3]); p += 4
            for v in (l, r - 1):
                for h in range(LOG, 0, -1):
                    u = v >> h
                    if co[u]:
                        z = lz[u]
                        c = 2 * u; t[c] = z; lz[c] = z; co[c] = True
                        c += 1; t[c] = z; lz[c] = z; co[c] = True
                        co[u] = False
            l0 = l; r0 = r
            while l < r:
                if l & 1:
                    t[l] = x; lz[l] = x; co[l] = True; l += 1
                if r & 1:
                    r -= 1; t[r] = x; lz[r] = x; co[r] = True
                l >>= 1; r >>= 1
            for v in (l0, r0 - 1):
                v >>= 1
                while v:
                    if not co[v]:
                        a = t[2 * v]; b = t[2 * v + 1]

```

```

        t[v] = a if a > b else b
        v >>= 1
    else:
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        p += 3
        for v in (l, r - 1):
            for h in range(LOG, 0, -1):
                u = v >> h
                if co[u]:
                    z = lz[u]
                    c = 2 * u; t[c] = z; lz[c] = z; co[c] = True
                    c += 1; t[c] = z; lz[c] = z; co[c] = True
                    co[u] = False
            res = NEG
        while l < r:
            if l & 1:
                if t[l] > res: res = t[l]
                l += 1
            if r & 1:
                r -= 1
                if t[r] > res: res = t[r]
            l >>= 1; r >>= 1
        ghi(res)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài F. Vị trí đầu tiên đủ lớn

```

# Problem F: first index i with a_i >= x, plus point assignment.
# The slow way is a binary search over the answer with a range-max query inside it,
# O(log^2 n). The way worth learning is to DESCEND the tree, O(log n): the root already
# knows the maximum of the whole array, so if it is below x the answer is -1; otherwise
# walk down and always step into the left child when that child can still contain a
# large enough element. The first leaf reached is the smallest index that works.
# Padding leaves hold minus infinity, so the walk never strays into them.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    size = 1
    while size < n:
        size <= 1
    NEG = -(1 << 62)
    t = [NEG] * (2 * size)
    for i in range(n):
        t[size + i] = int(data[p]); p += 1
    for v in range(size - 1, 0, -1):
        a = t[2 * v]; b = t[2 * v + 1]
        t[v] = a if a > b else b

    out = []
    ghi = out.append

```

```

for _ in range(q):
    loai = data[p]
    if loai == b'1':
        i = int(data[p + 1]); v = int(data[p + 2]); p += 3
        j = size + i - 1
        t[j] = v
        j >>= 1
        while j:
            a = t[2 * j]; b = t[2 * j + 1]
            moi = a if a > b else b
            if t[j] == moi:
                break
            t[j] = moi
            j >>= 1
    else:
        x = int(data[p + 1]); p += 2
        if t[1] < x:
            ghi(-1)
        else:
            v = 1
            while v < size:
                v <<= 1
                if t[v] < x:      # the left child cannot hold it, go right
                    v += 1
            ghi(v - size + 1)

sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài G. Đoạn con liên tiếp có tổng lớn nhất

```

# Problem G: maximum sum of a non-empty contiguous block, plus point assignment.
# One number per node is not enough. When two neighbouring segments are joined, the best
# block of the union is either wholly on the left, wholly on the right, or it STRADDLES
# the boundary, and the third case needs the best suffix of the left part and the best
# prefix of the right part. So a node carries four numbers: total sum, best prefix, best
# suffix, best block.
#
# Because the block must be non-empty, prefix, suffix and best all start at minus
# infinity rather than 0, and the answer for an all-negative range is its largest
# element. The neutral element for joining is (sum 0, the other three minus infinity).
#
# Python note. Four parallel lists beat a list of 4-tuples here: building a tuple per
# node allocates, and allocation is the expensive part of an inner loop in CPython.
# Unlike max or gcd, this join is NOT commutative, so the bottom-up query has to keep
# the pieces taken from the left end and the pieces taken from the right end apart and
# only put them together at the very end.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    size = 1
    while size < n:
        size <<= 1

```

```

NEG = -(1 << 62)
tong = [0] * (2 * size)
pre = [NEG] * (2 * size)
suf = [NEG] * (2 * size)
tot = [NEG] * (2 * size)

for i in range(n):
    x = int(data[p]); p += 1
    j = size + i
    tong[j] = pre[j] = suf[j] = tot[j] = x
for v in range(size - 1, 0, -1):
    a = 2 * v; b = a + 1
    tong[v] = tong[a] + tong[b]
    x = tong[a] + pre[b]; pre[v] = pre[a] if pre[a] > x else x
    x = tong[b] + suf[a]; suf[v] = suf[b] if suf[b] > x else x
    m = tot[a] if tot[a] > tot[b] else tot[b]
    x = suf[a] + pre[b]
    tot[v] = m if m > x else x

out = []
ghi = out.append
for _ in range(q):
    loai = data[p]
    if loai == b'1':
        i = int(data[p + 1]); val = int(data[p + 2]); p += 3
        j = size + i - 1
        tong[j] = pre[j] = suf[j] = tot[j] = val
        j >>= 1
        while j:
            a = 2 * j; b = a + 1
            tong[j] = tong[a] + tong[b]
            x = tong[a] + pre[b]; pre[j] = pre[a] if pre[a] > x else x
            x = tong[b] + suf[a]; suf[j] = suf[b] if suf[b] > x else x
            m = tot[a] if tot[a] > tot[b] else tot[b]
            x = suf[a] + pre[b]
            tot[j] = m if m > x else x
            j >>= 1
    else:
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        p += 3
        # Left accumulator and right accumulator, both starting neutral.
        lts, ltp, ltf, ltb = 0, NEG, NEG, NEG
        rts, rtp, rtf, rtb = 0, NEG, NEG, NEG
        while l < r:
            if l & 1:
                s2 = tong[l]; p2 = pre[l]; f2 = suf[l]; b2 = tot[l]
                x = lts + p2; np = ltp if ltp > x else x
                x = s2 + ltf; nf = f2 if f2 > x else x
                m = ltb if ltb > b2 else b2
                x = ltf + p2
                ltb = m if m > x else x
                ltp = np; ltf = nf; lts = lts + s2
                l += 1
            if r & 1:
                r -= 1
                s2 = tong[r]; p2 = pre[r]; f2 = suf[r]; b2 = tot[r]
                x = s2 + rtp; np = p2 if p2 > x else x
                x = rts + f2; nf = rtf if rtf > x else x
                m = b2 if b2 > rtb else rtb
                x = f2 + rtp
                rtb = m if m > x else x
                rtp = np; rtf = nf; rts = s2 + rts
            l >>= 1; r >>= 1

```

```

        # Finally join left accumulator with right accumulator, in that order.
        m = ltb if ltb > rtb else rtb
        x = ltf + rtp
        ghi(m if m > x else x)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài H. Cộng đoạn, giá trị nhỏ nhất trên đoạn

```

# Problem H: add on a range, minimum on a range.
# The tag is exactly the one from problem C, and the statement says one line has to be
# written differently. That line is the one that applies a tag to a node. For a SUM the
# node gains x times the width of its segment; for a MINIMUM it gains x, full stop,
# because adding the same x to every element of a segment shifts the minimum by x. The
# width factor left in by mistake is invisible on width-one segments.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    LOG = max(1, (n - 1).bit_length())
    size = 1 << LOG
    POS = 1 << 62
    t = [POS] * (2 * size)      # padding leaves must never win a minimum
    lz = [0] * (2 * size)
    for i in range(n):
        t[size + i] = int(data[p]); p += 1
    for v in range(size - 1, 0, -1):
        a = t[2 * v]; b = t[2 * v + 1]
        t[v] = a if a < b else b

    out = []
    ghi = out.append
    for _ in range(q):
        loai = data[p]
        if loai == b'1':
            l = int(data[p + 1]) - 1 + size
            r = int(data[p + 2]) + size
            x = int(data[p + 3]); p += 4
            for v in (l, r - 1):
                for h in range(LOG, 0, -1):
                    u = v >> h
                    z = lz[u]
                    if z:
                        c = 2 * u; t[c] += z; lz[c] += z
                        c += 1; t[c] += z; lz[c] += z
                        lz[u] = 0
            l0 = l; r0 = r
            while l < r:
                if l & 1:
                    t[l] += x; lz[l] += x; l += 1
                if r & 1:
                    r -= 1; t[r] += x; lz[r] += x
            l >>= 1; r >>= 1
        else:
            ghi(int(data[p]))
            p += 1

```

```

        for v in (l0, r0 - 1):
            v >>= 1
            while v:
                a = t[2 * v]; b = t[2 * v + 1]
                t[v] = (a if a < b else b) + lz[v]
                v >>= 1
    else:
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        p += 3
        for v in (l, r - 1):
            for h in range(LOG, 0, -1):
                u = v >> h
                z = lz[u]
                if z:
                    c = 2 * u; t[c] += z; lz[c] += z
                    c += 1; t[c] += z; lz[c] += z
                    lz[u] = 0
        res = POS
        while l < r:
            if l & 1:
                if t[l] < res: res = t[l]
                l += 1
            if r & 1:
                r -= 1
                if t[r] < res: res = t[r]
            l >>= 1; r >>= 1
        ghi(res)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```

Bài I. Giá trị lớn nhất và số lần xuất hiện

```

# Problem I: maximum on a range together with how many times it occurs.
# Joining two nodes has THREE cases, not two: left bigger, right bigger, and equal. Miss
# the third and the maximum still comes out right every single time while the count is
# silently too small, which is why this bug survives casual testing. Padding leaves hold
# minus infinity with a count of zero, so they can never tie with a real element.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    size = 1
    while size < n:
        size <<= 1
    NEG = -(1 << 62)
    mx = [NEG] * (2 * size)
    dem = [0] * (2 * size)
    for i in range(n):
        mx[size + i] = int(data[p]); dem[size + i] = 1; p += 1
    for v in range(size - 1, 0, -1):
        a = 2 * v; b = a + 1
        if mx[a] > mx[b]:

```



```

        mx[v] = mx[a]; dem[v] = dem[a]
    elif mx[a] < mx[b]:
        mx[v] = mx[b]; dem[v] = dem[b]
    else:
        mx[v] = mx[a]; dem[v] = dem[a] + dem[b]

out = []
ghi = out.append
for _ in range(q):
    loai = data[p]
    if loai == b'1':
        i = int(data[p + 1]); val = int(data[p + 2]); p += 3
        j = size + i - 1
        mx[j] = val; dem[j] = 1
        j >>= 1
        while j:
            a = 2 * j; b = a + 1
            if mx[a] > mx[b]:
                m, c = mx[a], dem[a]
            elif mx[a] < mx[b]:
                m, c = mx[b], dem[b]
            else:
                m, c = mx[a], dem[a] + dem[b]
            if mx[j] == m and dem[j] == c:
                break
            mx[j] = m; dem[j] = c
            j >>= 1
    else:
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        p += 3
        m = NEG; c = 0
        while l < r:
            if l & 1:
                if mx[l] > m:
                    m = mx[l]; c = dem[l]
                elif mx[l] == m:
                    c += dem[l]
                l += 1
            if r & 1:
                r -= 1
                if mx[r] > m:
                    m = mx[r]; c = dem[r]
                elif mx[r] == m:
                    c += dem[r]
            l >>= 1; r >>= 1
        ghi("%d %d" % (m, c))

sys.stdout.write('\n'.join(out) + '\n')

main()

```

Bài J. Đảo bit trên đoạn, đếm số 1

```

# Problem J: flip every bit on a range, count the ones on a range.
# The tag is of a third kind again. Flipping is its own inverse, so a tag is a single
# bit and composing two tags is an exclusive or: flipping twice is doing nothing. That
# is why the tag here is never "set to 1", it is "flip", and why lz[c] ^= 1 rather than
# lz[c] = 1. Applying the tag to a node turns its count of ones into width minus count.

```

```

# Padding leaves have width zero, so flipping them keeps them at zero.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    p = 0
    n = int(data[p]); q = int(data[p + 1]); p += 2

    LOG = max(1, (n - 1).bit_length())
    size = 1 << LOG
    t = [0] * (2 * size)      # how many ones in the segment
    lz = [0] * (2 * size)    # 1 means the children still owe a flip
    w = [0] * (2 * size)

    for i in range(n):
        t[size + i] = int(data[p]); w[size + i] = 1; p += 1
    for v in range(size - 1, 0, -1):
        t[v] = t[2 * v] + t[2 * v + 1]
        w[v] = w[2 * v] + w[2 * v + 1]

    out = []
    ghi = out.append
    for _ in range(q):
        loai = data[p]
        l = int(data[p + 1]) - 1 + size
        r = int(data[p + 2]) + size
        p += 3
        for v in (l, r - 1):
            for h in range(LOG, 0, -1):
                u = v >> h
                if lz[u]:
                    c = 2 * u; t[c] = w[c] - t[c]; lz[c] ^= 1
                    c += 1; t[c] = w[c] - t[c]; lz[c] ^= 1
                    lz[u] = 0
        if loai == b'1':
            l0 = l; r0 = r
            while l < r:
                if l & 1:
                    t[l] = w[l] - t[l]; lz[l] ^= 1; l += 1
                if r & 1:
                    r -= 1; t[r] = w[r] - t[r]; lz[r] ^= 1
                l >>= 1; r >>= 1
            for v in (l0, r0 - 1):
                v >>= 1
                while v:
                    s = t[2 * v] + t[2 * v + 1]
                    t[v] = (w[v] - s) if lz[v] else s
                    v >>= 1
            else:
                s = 0
                while l < r:
                    if l & 1:
                        s += t[l]; l += 1
                    if r & 1:
                        r -= 1; s += t[r]
                    l >>= 1; r >>= 1
            ghi(s)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()

```